# Практическая работа №4

Интеграция Flask и SQLite: чтение данных

Выполнил: Севостьянов Роман Андреевич

Группа: ЗИСП11-43

Репозиторий: https://github.com/OborMOT2006/practies-py

## Цель и краткое описание

Страница получает записи из таблицы tasks через SELECT. Функция get_connection() задаёт sqlite3.Row, после чего Jinja2 выводит поля задачи и её статус. Выполненные задачи зачёркиваются.

## Использованные технологии и файлы

Технологии: Python 3, Flask, SQLite, Jinja2, HTML/CSS

Исходные файлы в папке src: app.py, templates/index.html, app.db, init_db.py

## Основные проверки

- В браузере отображаются задачи из базы
- Изменение базы отражается при обновлении страницы
- Выполненная задача показана зачёркнутой

## Примеры запросов и ответов

GET / → 200 OK: HTML-список задач

SELECT id, title, status FROM tasks → Несколько записей из SQLite

## Результаты и изображения

Рисунок 1. 01 интерфейс



## Ответы на контрольные вопросы

1. Зачем Jinja2 вместо HTML-строки?

Удобнее хранить HTML отдельно от Python и вставлять данные через шаблон.

2. Чем отличаются {{ ... }} и {% ... %}?

Двойные фигурные скобки выводят значение, а скобки с процентами используются для циклов и условий.

3. Что делает render_template?

Берёт HTML-файл, подставляет туда items и возвращает готовую страницу браузеру.

4. Почему при SELECT не нужен commit()?

Потому что мы не меняем записи, а только читаем.

5. Что такое sqlite3.Row?

Это формат строки, с которым можно обращаться к колонкам по имени, например item.title.

6. Где Flask ищет HTML?

По умолчанию в папке templates рядом с app.py. Это можно поменять настройкой template_folder.

7. Что будет при пустом items?

Сработает ветка else, и появится сообщение о пустом списке.

## Вывод

Задачи практической работы выполнены. Код и база данных находятся в папке src, результаты проверки — в папке screenshots.